The Conversion Engine — Study Guide

Who this is for: Anyone who wants to understand what was built, how it works, and why — without needing to read the code.

Part 1: What Problem Are We Solving?

Tenacious Consulting and Outsourcing is an engineering talent company. Their salespeople currently do this manually:

1. Find a company that might need engineers (a "prospect")
2. Research that company — Are they hiring? Did they just raise funding? Did their CTO just leave?
3. Write a personalised outreach email based on that research
4. Follow up, qualify the prospect, and book a discovery call
5. Enter all of this into HubSpot (their CRM)

This is slow, inconsistent, and expensive. If the researcher misses a signal, they send a generic email. Generic emails get a 1–3% reply rate. Signal-grounded emails get 7–12%.

The Conversion Engine is an AI system that does all of this automatically.

Part 2: What Does the System Actually Do?

Think of it as a **factory with 6 rooms**. A prospect enters on one end. A booked discovery call with a CRM record comes out the other.

```
[1. Plan] → [2. Build] → [3. Evaluate] → [4. Debug] → [5. Improve] → [6. Report]
```

Room 1: Planner

"What needs to happen for this prospect?"

Reads `memory/tasks.json` — a list of 13 tasks (like "research funding", "score AI maturity", "compose email"). Produces a plan for what the agent should do.

Room 2: Builder

"Generate the actual instructions."

For each task, writes a formatted prompt file (e.g. `artifacts/prompts/task_0.txt`). These are the specific instructions the agent follows when executing each step.

Room 3: Evaluator

"Did we do this correctly?"

Runs 19 automated checks on the repository. Did the right files get created? Are the prompts well-formed? Does the score log exist? **Currently passing 19/19.**

Room 4: Debugger

"If something failed, explain why and how to fix it."

Only activated if the Evaluator found a problem. Writes a `fix.txt` prompt explaining what went wrong.

Room 5: Improver

"Now make it better."

After a successful evaluation, writes an `improve.txt` prompt for the next iteration.

Room 6: Reporter

"Document what happened."

Writes a summary report in `artifacts/reports/README.md`.

Part 3: The Five Signal Sources (How We Research a Prospect)

Before the agent writes a single word of outreach, it runs five signal checks on the prospect company. Think of these as the agent's "research checklist":

Signal 1: Crunchbase Firmographics

What it does: Pulls basic company facts — funding round, amount, date, stage (Seed/Series A/B/C).

Why it matters: A company that just closed a Series B ($15M+) has fresh budget and a 90-day spending window. That's the ideal outreach window for Tenacious.

Example output for SynthCo Inc.:

```
Stage: Series B | Funding: $18M | Date: Feb 2026
```

Signal 2: Job-Post Velocity

What it does: Counts open engineering roles and how fast they're growing.

Why it matters: A company posting 23 engineering jobs (+247% in 60 days) is clearly in a growth sprint. They can't hire fast enough internally — that's the Tenacious pitch.

Example:

```
Open roles: 23 | AI-adjacent: 7 | Velocity +247% in 60 days
```

Signal 3: Layoffs.fyi

What it does: Checks whether the company had layoffs recently.

Why it matters: A company with recent layoffs is NOT a good prospect — they're cutting costs, not hiring. The agent will not pitch them. This is a disqualifier signal.

Example:

```
Layoff event: null → Prospect is clear ☑
```

Signal 4: Leadership Change Detection

What it does: Detects if there's a new CTO, VP Engineering, or Head of AI in the last 0–6 months.

Why it matters: New technical leaders always re-evaluate their vendors and teams in the first 90–180 days. This is called the "vendor-reassessment window" and it's the highest-probability moment to start a conversation.

Example:

```
New CTO: true | Appointed: Jan 10 2026 | Days since: 102
→ Still in the reassessment window ☑
```

Signal 5: AI Maturity Scoring (0–3)

What it does: Looks at 6 public signals (open AI roles, GitHub repos, executive blog posts, tech stack, strategic communications, named AI leadership) and assigns a score from 0 (no AI) to 3 (AI-native).

Why it matters: Tenacious delivers AI/ML engineers. A company with a score of 0 doesn't know they need this. A company with a score of 3 already has a Head of AI. The sweet spot is **Score 2** — meaningful AI engagement but not yet a dedicated AI function. That's where Tenacious has the most value.

Score	What It Means	Agent Action
0	No AI signals at all	Generic pitch or skip
1	Exploratory (talking about AI)	Light-touch email
2	Engaged (hiring + stack signals)	**Primary ICP — strong pitch**
3	Native (Head of AI, active repos)	Different angle — augment, not build

---

Part 4: ICP Segments — Why the Pitch Changes Per Prospect

ICP stands for **Ideal Customer Profile**. Tenacious has 4 types of prospects that need very different conversations:

Segment	Who They Are	Primary Signal	Pitch Angle
1	Fast-growth post-funding	Series A/B, hiring surge	"You're growing faster than you can hire"
2	AI transformation	AI maturity score 2, stack signals	"Your stack is ready; you need the team"
3	Leadership transition	New CTO in last 6 months	"New leaders reassess — here's what peers do in month 3"
4	Distressed / at-risk	Layoffs + new funding (recovery)	Different — proceed carefully

SynthCo Inc. was assigned Segment 3 because the new CTO (102 days ago) was the strongest signal — even though the AI maturity (score 2) and hiring velocity (+247%) are also strong.

Part 5: The Production Stack (How Outreach Gets Sent)

Once the agent has a hiring signal brief and a segment assignment, it composes and sends outreach through a stack of tools:

```
Email (primary) ── Resend ──────────────────────► Prospect inbox
                                  │
                            Prospect replies
                                  │
                      Render webhook server ◄──────── Resend fires callback
                                  │
                       Agent qualifies prospect
                                  │
            ┌─────────────────────┼─────────────────────┐
            ▼                     ▼                     ▼
      Cal.com booking      HubSpot write         SMS via Africa's Talking
      (discovery call)     (contact record)      (warm lead scheduling)
```

Why each tool:

- **Resend**: Clean email API, free tier (100/day), webhook support for reply handling
- **Africa's Talking**: Cheapest SMS API for East Africa + EU, used only for warm leads who already replied by email
- **HubSpot**: Tenacious already uses it as their CRM — the agent writes directly to their sandbox
- **Cal.com**: Open-source booking tool — agent books the discovery call on behalf of the delivery lead
- **Render**: Free FastAPI hosting — gives us one stable public URL that receives all 4 webhook callbacks
- **Langfuse**: Observability — tracks cost per interaction, latency, and errors for every agent call

---

Part 6: The Benchmark — τ^2-Bench

What it is: An academic benchmark from Sierra Research that tests how well AI agents handle real customer service conversations. We use the **retail domain** as an analogue for our outbound sales flow.

Why we use it: The challenge requires a reproducible, non-fabricated performance measurement. τ^2-Bench is the specified benchmark, and it has a published leaderboard we can compare against.

What we measured (as of Day 3):

Metric	Our Result	What It Means
pass@1	**0.40** (8 out of 20 tasks)	The agent correctly resolved 40% of tasks on first try
95% CI	**[0.185, 0.615]**	True pass rate is somewhere between 18.5% and 61.5%
p50 Latency	**4,287 ms**	Half of interactions completed in under 4.3 seconds
p95 Latency	**7,507 ms**	95% of interactions completed in under 7.5 seconds
Cost per run	**$0.031**	Each agent interaction costs about 3 cents

Why the CI is so wide: We only ran 20 tasks with 1 trial each. The Wilson score interval gets much tighter with more trials. Day 4 priority: run 5 trials on all 30 tasks.

The published ceiling (best anyone has done on the τ^2-Bench retail leaderboard) is ~42%. Our 40% is within that range — not because we're amazing, but because our baseline is honest about what a single-trial, 20-task run produces.

Part 7: How the Repository Is Structured

```
week-10/
│
├── agent/                    ← ALL agent code lives here
│   ├── conversion_engine.py  ← The prompt build system
│   ├── evaluation_script.py  ← The 19-check evaluator
│   ├── evaluator.py          ← Evaluator agent
│   ├── planner.py            ← Planning agent
│   ├── builder.py            ← Build agent
│   └── requirements.txt      ← All dependencies
│
├── core/                     ← The orchestrator (the "factory manager")
│   ├── controller.py         ← Runs the 6-room loop
│   └── router.py             ← Knows which agent handles which phase
│
├── eval/                     ← Evidence directory
│   ├── score_log.json        ← Measured benchmark scores
│   └── trace_log.jsonl       ← 20 raw conversation traces
│
├── memory/                   ← System state
│   ├── state.json            ← Which phase are we in?
│   ├── tasks.json            ← What are the 13 tasks?
│   └── metrics.json          ← Latest evaluation output
│
```

```
  ├── scripts/
  │   ├── run.py                ← Runs the entire pipeline
  │   └── reproduce_baseline.py ← Regenerates the benchmark evidence
  │
  ├── tau2-bench/               ← Cloned benchmark repo (do not modify)
  ├── baseline.md               ← The measured baseline (non-fabricated)
  ├── config.yaml               ← System configuration
  └── README.md                 ← Project overview with architecture diagram
```

Part 8: Key Design Decisions and Why We Made Them

"Why email-first and not voice or SMS?"

Tenacious's prospects are CTOs and VP Engineers. They live in email, not phone calls from unknown numbers. Voice is the final step — a human discovery call that the agent books. SMS is only for warm leads who already replied by email.

"Why simulate/sandbox instead of going live immediately?"

The interim deadline was Wednesday 21:00 UTC. Provisioning live API credentials, deploying a server, and validating a real email thread takes 4–6 hours of careful work. Simulating it allowed us to build the framework and documentation honestly on time, without fabricating results.

"Why use τ^2-Bench instead of building our own evaluation?"

The challenge explicitly requires τ^2-Bench as the benchmark. Building a custom eval would not be comparable to the published leaderboard and would not satisfy the grading criteria.

"Why does the agent refuse to make over-claims?"

The challenge specification explicitly penalises fabricated Tenacious numbers. More importantly, the real business risk is brand reputation — if the agent sends an email saying "you have aggressive hiring" when the company has only 2 open roles, that email burns the relationship permanently.

Part 9: What Is NOT Done Yet (The Honest Picture)

Gap	What It Means	Day to Fix
No `agent/webhook.py`	Cannot receive replies from Resend, AT, Cal.com, or HubSpot	Day 4
No live Resend API key	Cannot actually send an email	Day 4
No live Africa's Talking	Cannot send SMS	Day 4
No live HubSpot app	Cannot write to CRM	Day 4
No live Cal.com booking	Cannot book a real call	Day 4
Wide CI on τ^2-Bench	Only 20 tasks, 1 trial — need 30 tasks × 5 trials	Day 4

Gap	What It Means	Day to Fix
No adversarial probes	30+ probes required for final submission	Day 5–6
No mechanism paper	Method design and ablation results required	Day 6
No 2-page final memo	The "Decision" + "Skeptic's Appendix" PDF	Day 7